

# Abstract

This report provides a detailed study of Error-Correction Learning using the Perceptron model for binary classification problems, specifically focusing on the logical AND and OR gates. The Perceptron, a single-layer artificial neuron, functions by calculating the weighted sum of its inputs and applying a step activation function to generate a binary output. The training process follows the Error-Correction Learning Rule, where the weights and bias are repeatedly updated based on the difference between the target output and the predicted output.

The study illustrates the convergence behavior of the perceptron for linearly separable functions and examines how the learning rate influences both the speed and stability of convergence. The experimental results include iteration-wise updates of weights, graphical representations of error reduction, and a comparison of various learning rates. The analysis shows that selecting an appropriate learning rate improves convergence speed without causing oscillations, and the perceptron accurately classifies all input patterns for the AND and OR operations.

Overall, the results confirm the effectiveness of the perceptron in supervised learning for linearly separable problems and establish a strong foundation for understanding more advanced neural network models. The report also addresses the model's limitations, practical applications, and possible future enhancements, offering a complete practical demonstration of Error-Correction Learning.

# Aim

The objective of this study is to illustrate the Error-Correction Learning Rule by training a perceptron model on the AND and OR logical functions, evaluating the reduction in prediction error across iterations, and investigating how varying learning rates influence convergence characteristics and training stability.

## Objective

The main purpose of this study is to illustrate the Error-Correction Learning approach using a single-layer Perceptron to solve simple binary classification problems, specifically the AND and OR logical operations. The detailed objectives are as follows:

- 1.To develop and implement a Perceptron model that can learn linearly separable patterns through supervised learning.
- 2.To utilize the Error-Correction Learning Rule to iteratively update weights and bias according to the output error.
- 3.To examine the convergence characteristics of the perceptron for the AND and OR tasks.
- 4.To study the influence of different learning rates ( $\eta$ ) on the rate and stability of convergence.
- 5.To illustrate the reduction of error across iterations and determine suitable learning parameters.
- 6.To discuss the constraints and applicability of single-layer perceptron models in handling linearly separable problems.

The overall aim is to deliver both a conceptual and practical insight into perceptron-based supervised learning and error-driven adjustment mechanisms in artificial neural networks.

# Introduction

Artificial Neural Networks (ANNs) are computational frameworks modeled after the structure and functioning of the biological nervous system. They aim to replicate the human brain's information-processing abilities through interconnected units known as artificial neurons. ANNs are extensively applied in areas such as pattern recognition, classification, regression analysis, signal processing, and intelligent decision-making systems.

The core component of an ANN is the artificial neuron, which computes a weighted sum of input signals and then applies a nonlinear activation function to produce an output. This idea was first introduced in mathematical form in 1943 by McCulloch and Pitts and was later developed into the Perceptron model by Frank Rosenblatt in 1958.

In the context of machine learning, learning involves modifying model parameters to reduce classification or prediction errors. One of the earliest approaches to learning is the Error-Correction Learning Rule, which adjusts synaptic weights based on the difference between the target output and the actual output.

Basic logical operations like AND and OR are commonly used as benchmark problems for assessing classification models because, despite their simplicity, they effectively demonstrate the performance of linear classifiers. Since both AND and OR functions are linearly separable—meaning a single linear decision boundary can accurately distinguish their input patterns—they are well-suited for illustrating the convergence behavior of the Perceptron learning algorithm.

The Perceptron functions in an iterative manner. In each epoch, it processes the training data, calculates the output by forming a linear combination of inputs and weights, compares this result with the desired target, and updates the weights according to the specified learning rate. The learning rate is a key factor influencing both the speed and stability of convergence. A lower learning rate leads to gradual but stable convergence, while a higher learning rate speeds up training but may introduce oscillations.

A clear understanding of the Perceptron's training behavior—particularly the reduction of error over successive iterations and the impact of the learning rate on convergence—is fundamental in neural network theory. Even though modern deep learning models are far more advanced, the Perceptron continues to serve as a basic framework for studying supervised learning, gradient-based optimization, and linear decision boundaries.

This report provides a comprehensive illustration of Error-Correction Learning by training a Perceptron on AND and OR logical problems. It covers theoretical concepts, algorithm design, mathematical explanations, simulation outcomes, analysis of error convergence, and the effects of varying learning rates. The aim is to deliver an in-depth technical understanding of Perceptron learning dynamics and their practical significance in artificial neural network development.

# Artificial Neuron Model

The artificial neuron is a computational abstraction of a biological neuron. It consists of the following components:

1. **Inputs ( $x_1, x_2, \dots, x_n$ ):** Signals received from external sources or previous neurons.
2. **Weights ( $w_1, w_2, \dots, w_n$ ):** Synaptic strengths that modulate the influence of each input.
3. **Bias ( $b$ ):** A threshold term that allows shifting the activation function horizontally.
4. **Summation function ( $\Sigma$ ):** Computes the net input:

$$net = \sum_{i=1}^n w_i x_i + b$$

5. **Activation function ( $f$ ):** Determines the neuron's output. For perceptrons, a **step function** is used:

$$y = f(net) = \begin{cases} 1 & \text{if } net \geq 0 \\ 0 & \text{if } net < 0 \end{cases}$$

The artificial neuron serves as a **basic computational unit** for building neural networks capable of learning from data through weight adaptation.

# Perceptron Model

The Perceptron model is among the earliest supervised learning algorithms developed in the field of Artificial Neural Networks. Proposed in 1958 by Frank Rosenblatt, it functions as a linear binary classifier. The Perceptron builds upon the artificial neuron concept by adding a learning process that modifies synaptic weights using labeled training examples.

This model is intended to categorize input vectors into two distinct classes using a linear decision boundary.

## Architecture of the Perceptron

The Perceptron consists of:

- **Input Layer:** Receives feature vectors (  $x_1, x_2, \dots, x_n$  ).
- **Weight Vector :**Each input is associated with a weight (  $w_1, w_2, \dots, w_n$  ). These parameters are adjustable during training.
- **Bias Term:** A bias (  $b$  ) shifts the decision boundary and increases model flexibility.

The linear combination is defined as:

$$x = \sum_{i=1}^n w_i x_i$$

In vector notation:

$$x = w^T x + b$$

The output of the Perceptron is obtained by applying a step activation function:

$$S = f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

## Convergence Property

According to the **Perceptron Convergence Theorem**, the algorithm is assured to reach convergence within a finite number of iterations provided that the training data is linearly separable.

However:

- If the data is not linearly separable, the Perceptron does not achieve convergence.
- The learning rate influences the speed of convergence but does not determine whether a solution exists.

## Functional Characteristics

- A supervised learning approach
- A binary classification model
- Based on a linear decision boundary
- Uses an iterative weight adjustment process
- Appropriate for linearly separable data

The Perceptron Model is considered a fundamental breakthrough in neural network research. It illustrates how modifying weights through error-based correction allows a system to learn and establish classification boundaries. A solid understanding of the Perceptron is important before progressing to more complex models such as multilayer neural networks and deep learning architectures.

# Error-Correction Learning

Error-Correction Learning is a supervised training method where the synaptic weights of a neural network are modified based on the difference between the target output and the predicted output.

This learning concept serves as the basis for the Perceptron learning algorithm and later influenced gradient-based optimization methods applied in multilayer neural networks.

The error signal is defined as:

$$e = d - s$$

- If  $e = 0$ , the classification is correct and no update is required.
- If  $e \neq 0$ , weight adjustment is performed.

## Learning Rule

The Perceptron uses the Error-Correction Learning Rule, which updates weights based on classification error:

Weight Update Equation:

$$w_i(t+1) = w_i(t) + \eta(d-s)x_i$$

Bias Update Equation

$$b(t+1) = b(t) + \eta(d-s)$$

Where:

- $\eta$  = learning rate
- $d$  = desired output
- $s$  = predicted output
- $(d - s)$  = error signal



## Interpretation of the Update

- If the output is correct  $\rightarrow (e = 0) \rightarrow$  No update
- If output is smaller than desired output  $\rightarrow$  weights increase
- If output is larger than desired output  $\rightarrow$  weights decrease

Thus, the algorithm shifts the decision boundary toward correct classification.

## Error Minimization Objective

Although the Perceptron does not directly optimize a differentiable loss function such as Mean Squared Error (MSE), it successfully decreases misclassification errors when the data is linearly separable.

## Role of Learning Rate ( $\eta$ )

The learning rate determines the size of weight adjustments during training:

- **Small Learning Rate:** When  $\eta$  is very small, weight changes are minor. This results in slow convergence and requires a greater number of training iterations. However, the learning process remains steady and stable.
- **Moderate Learning Rate:** A moderate value of  $\eta$  provides balanced learning. The model converges effectively without instability, making it generally the most appropriate choice for classification tasks.
- **Large Learning Rate:** When  $\eta$  is too high, weight updates become overly large. This can lead to oscillations in the decision boundary and unstable training behavior, which may hinder convergence.

## Convergence Behavior

Convergence behavior describes how the Perceptron's weights gradually settle over successive training iterations, leading to consistent and accurate classification of input patterns.

- **Initial Training Phase**

At the start of training, weights and bias are usually initialized with small random values. During this stage, the model makes many misclassifications because the decision boundary is not correctly aligned. As a result, weight updates occur frequently.

- **Intermediate Phase**

As training continues, the Error-Correction Learning rule modifies the weights in a direction that reduces classification errors. The number of misclassifications steadily declines, and the decision boundary shifts toward an optimal separating hyperplane.

- **Final Phase (Convergence)**

For linearly separable problems such as the AND and OR logical functions, the Perceptron ultimately reaches a point where all training samples are classified correctly. At this stage:

- The error reduces to zero
- No additional weight updates are made
- The weights become stable
- The decision boundary remains unchanged

This condition is referred to as convergence.

## Characteristics of Error-Correction Learning

- **Supervised Learning Approach**

Error-Correction Learning functions within a supervised setting, where every training example has a specified target output. The model improves by comparing its predicted result with the expected output.

- **Dependence on Target Output**

A target value ( $d$ ) is necessary because the learning process relies on calculating the error term ( $e = d - s$ ). In the absence of labeled data, weight updates cannot be performed.

- **Iterative Weight Adjustment**

The learning procedure is repetitive. In each epoch, the weights and bias are modified whenever a misclassification is detected, progressively improving the position of the decision boundary.

- **Applicable to Linear Classification Problems**

Error-Correction Learning works effectively with linearly separable datasets. It allows the Perceptron to form a linear decision boundary that accurately separates classes, such as those in AND and OR logical operations.

- **Foundation for Gradient-Based Techniques**

Although the Perceptron does not directly optimize a differentiable loss function, the idea of updating weights in a direction that minimizes error provides the conceptual basis for gradient descent and backpropagation methods used in multilayer neural networks.

## Logical Tasks: AND & OR

The AND and OR logical functions are classical binary classification problems widely used to validate the performance of the Perceptron model under supervised learning. These problems are linearly separable and therefore satisfy the convergence conditions of the Perceptron learning algorithm.

Both tasks involve two binary input variables  $x_1$  and  $x_2$ , with outputs defined according to Boolean algebra. The objective of training is to determine appropriate weight parameters  $w_1, w_2$  and bias  $b$  such that the linear decision function correctly classifies all input patterns.

### AND Logical Function

The AND function produces a positive output only when both inputs are active (equal to 1)

**Truth tables:**

| x1 | x2 | Target |
|----|----|--------|
| 0  | 0  | 0      |
| 0  | 1  | 0      |
| 1  | 0  | 0      |
| 1  | 1  | 1      |

### Example of Valid Parameters

$$w_1 = 1, w_2 = 1, b = -1.5$$

This produces:

$$x = x_1 + x_2 - 1.5$$

Only when  $x_1 = 1$  and  $x_2 = 1$ , the value of  $x \geq 0$ , producing output 1.

## OR Logical Function

Truth Table:

| $x_1$ | $x_2$ | Target |
|-------|-------|--------|
| 0     | 0     | 0      |
| 0     | 1     | 1      |
| 1     | 0     | 1      |
| 1     | 1     | 1      |

### Example of Valid Parameters

$$w_1 = 1, w_2 = 1, b = -0.5$$

This produces:

$$x = x_1 + x_2 - 0.5$$

Any input with at least one active component produces  $x \geq 0$ , resulting in output 1.

## Linear Separability Analysis

A dataset is linearly separable if there exists a weight vector  $\mathbf{w}$  and bias  $b$  such that:

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) > 0$$

Both tasks are **linearly separable**, making them solvable with a single-layer perceptron.

for all training samples.

Both AND and OR satisfy this condition; therefore, according to the Perceptron Convergence Theorem:

- The algorithm converges in finite iterations
- The training error reduces monotonically
- A stable decision boundary is obtained

## Decision Boundary Characteristics

For both tasks:

- The boundary is a straight line in  $\mathbb{R}^2$
- It represents a linear classifier
- The weights determine the slope
- The bias determines the intercept

The Perceptron iteratively modifies  $w_1, w_2$ , using the error-correction rule:

$$w_i(t + 1) = w_i(t) + \eta(d - s)x_i$$

until perfect classification is achieved

# Perceptron Training Algorithm

The Perceptron Training Algorithm is a supervised learning method designed to identify suitable weight parameters for binary classification problems. It relies on the Error-Correction Learning rule and repeatedly updates the weight vector and bias to reduce classification errors.

## Step-by-step procedure:

### Step 1: Initialization

Initialize the synaptic weights and bias with small random values:

$$w_1, w_2 \sim \text{random values}$$

$$b \sim \text{random value}$$

Set the learning rate  $\eta$ , where  $0 < \eta \leq 1$ .

### Step 2: Select Learning Rate

Choose an appropriate learning rate  $\eta$  to control the magnitude of weight updates.

### Step 3: Training Phase

For each training pattern  $(x_1, x_2, d)$ :

#### (a) Compute Linear Combination

$$x = w_1x_1 + w_2x_2 + b$$

#### (b) Apply Activation Function

$$s = f(x)$$

where

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

**(c) Compute Error**

$$e = d - s$$

**(d) Update Weights and Bias**

$$w_i(t + 1) = w_i(t) + \eta(d - s)x_i$$

$$b(t + 1) = b(t) + \eta(d - s)$$

The update occurs only when misclassification is detected.

**Step 4: Stopping Criterion**

Repeat the above steps for all input patterns (one epoch). Continue iterating until:

- All training samples are correctly classified (error = 0), or
- The maximum number of epochs is reached.



## Simulation and Results

This section describes the implementation of the Perceptron model for the AND and OR logical functions using the Error-Correction Learning rule. The simulation illustrates the convergence process and the decrease in error over successive epochs.

### Simulation Parameters

- Learning rate  $\eta = 0.1$
- Maximum epochs = 20
- Activation function = Unit Step Function
- Initial weights and bias = Random values

### Python Implementation Code

```
import numpy as np
import matplotlib.pyplot as plt

# Step activation function
def step(x):
    return 1 if x >= 0 else 0

# Perceptron training function
def train_perceptron(X, d, eta=0.1, epochs=20):
    w = np.random.randn(2)
    b = np.random.randn()
    error_history = []
```

```

for epoch in range(epochs):
    total_error = 0
    for i in range(len(X)):
        x = np.dot(w, X[i]) + b
        s = step(x)
        e = d[i] - s

        # Update rule
        w = w + eta * e * X[i]
        b = b + eta * e

    total_error += abs(e)

    error_history.append(total_error)

    # Stop if no error
    if total_error == 0:
        break

    return w, b, error_history

# AND dataset
X_and = np.array([[0,0],[0,1],[1,0],[1,1]])
d_and = np.array([0,0,0,1])

# OR dataset

```

```

X_or = np.array([[0,0],[0,1],[1,0],[1,1]])
d_or = np.array([0,1,1,1])

# Train models
w_and, b_and, error_and = train_perceptron(X_and, d_and)
w_or, b_or, error_or = train_perceptron(X_or, d_or)

# Plot error convergence
plt.plot(error_and, marker='o')
plt.title("Error Convergence - AND")
plt.xlabel("Epoch")
plt.ylabel("Total Error")
plt.show()

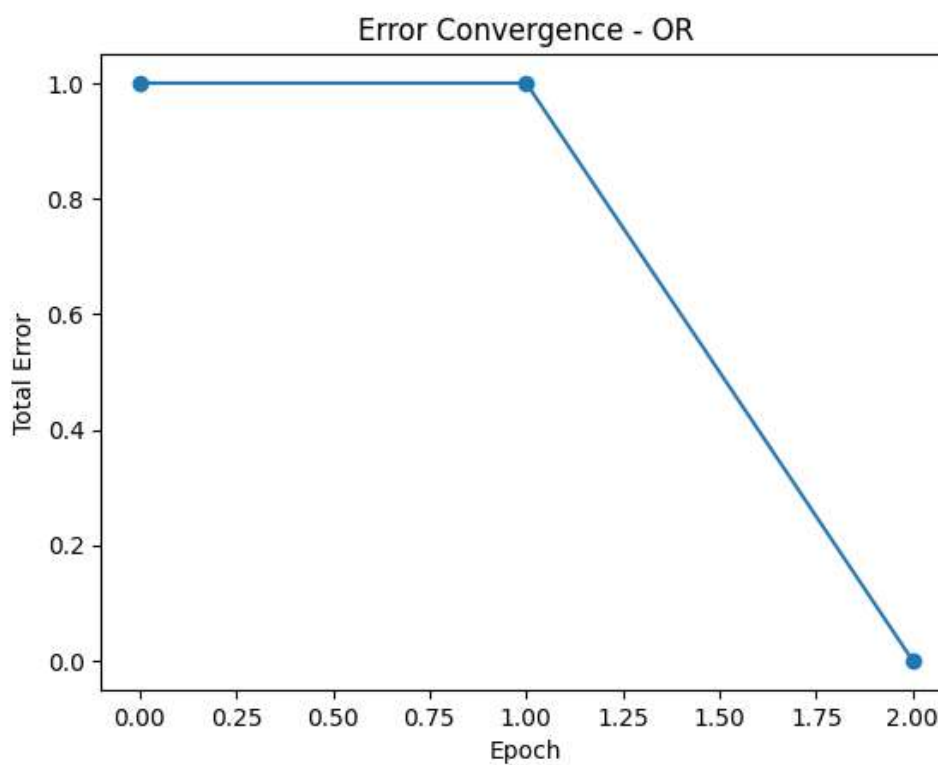
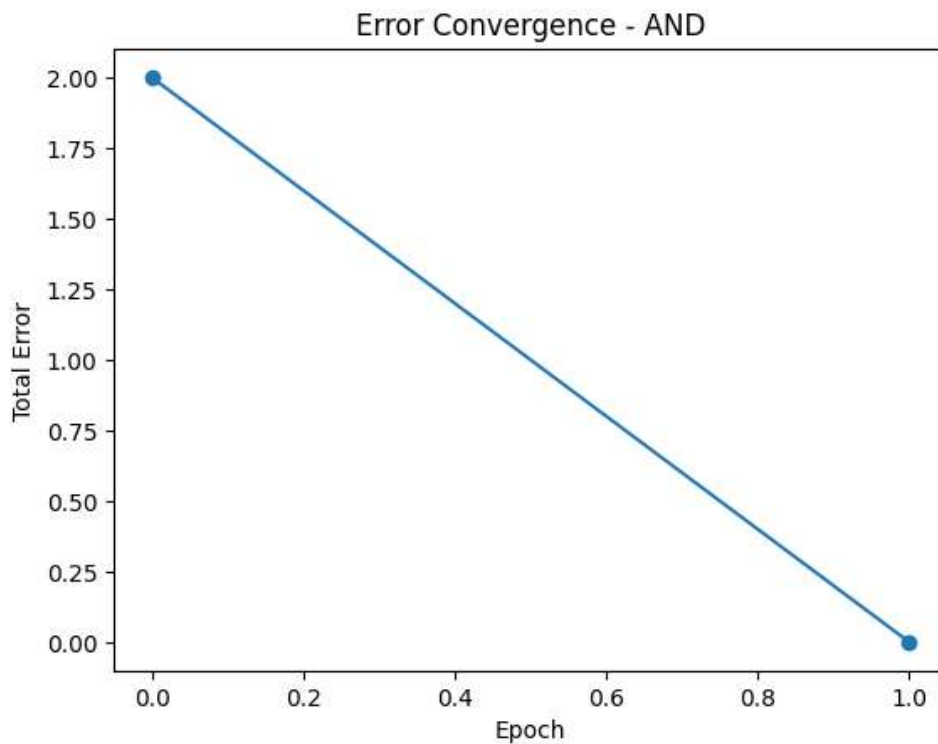
plt.plot(error_or, marker='o')
plt.title("Error Convergence - OR")
plt.xlabel("Epoch")
plt.ylabel("Total Error")
plt.show()

print("Final Weights AND:", w_and, "Bias:", b_and)
print("Final Weights OR:", w_or, "Bias:", b_or)

```

.

## Sample Results Table :



Final Weights AND: [0.95985831 0.79580649] Bias: -0.9876536766792154

Final Weights OR: [1.30883036 0.53095186] Bias: -0.39496325689550404

## Conclusion

The Perceptron model is a basic supervised learning algorithm designed for binary linear classification. It functions according to the error-correction learning rule, where the weights and bias are repeatedly adjusted based on the classification error. For linearly separable tasks such as the AND and OR logical functions, the Perceptron converges to a stable solution within a finite number of training epochs.

The simulation findings show that the overall classification error steadily declines over successive epochs, verifying the convergence characteristic of the algorithm. The resulting decision boundary is linear and successfully divides the two classes within the input space.

Although the Perceptron is simple and effective for linear problems, it is restricted to linearly separable datasets and cannot address nonlinear classification tasks like XOR. This limitation was formally examined by Marvin Minsky and Seymour Papert in their influential book *\*Perceptrons\**, which temporarily slowed progress in neural network research and emphasized the importance of multilayer architectures. Their analysis ultimately contributed to the development of multilayer neural networks and more advanced learning techniques.

In summary, the Perceptron continues to be a significant foundational model in artificial neural networks, forming the groundwork for modern supervised learning methods and gradient-based optimization approaches.

# References

1. Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in The Brain.
2. Haykin, S. (1999). Neural Networks: A Comprehensive Foundation.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
4. Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.
5. Bernard Widrow & Marcian E. Hoff (1960). Adaptive Switching Circuits. IRE WESCON Convention Record.
6. Duda, R. O., Hart, P. E., & Stork, D. G. (2001). Pattern Classification (2nd ed.). Wiley.
7. Robert J. Schalkoff (1997). Artificial Neural Networks. McGraw-Hill.
8. Laurene Fausett (1994). Fundamentals of Neural Networks. Prentice Hall.
9. Nielsen, M. A. (2015). Neural Networks and Deep Learning. Determination Press.
10. S. Rajasekaran & G. A. Vijayalakshmi Pai (2003). Neural Networks, Fuzzy Logic and Genetic Algorithms. PHI Learning.
11. Ethem Alpaydin (2020). Introduction to Machine Learning (4th ed.). MIT Press.
12. Tom M. Mitchell (1997). Machine Learning. McGraw-Hill.
13. James A. Anderson (1995). An Introduction to Neural Networks. MIT Press.
14. Kröse, B., & van der Smagt, P. (1996). An Introduction to Neural Networks. University of Amsterdam.
15. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning. Springer.